

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA0309

Register Number	1921260016
Name	Charumathi S R

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2 **EACH QUESTIONS CARRIES:** 10 MARKS **Total Marks:**20 **Duration:** ONE HOUR

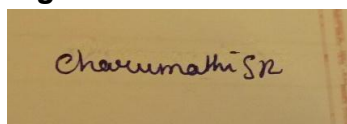
Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Charumathi S R(1921260016) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

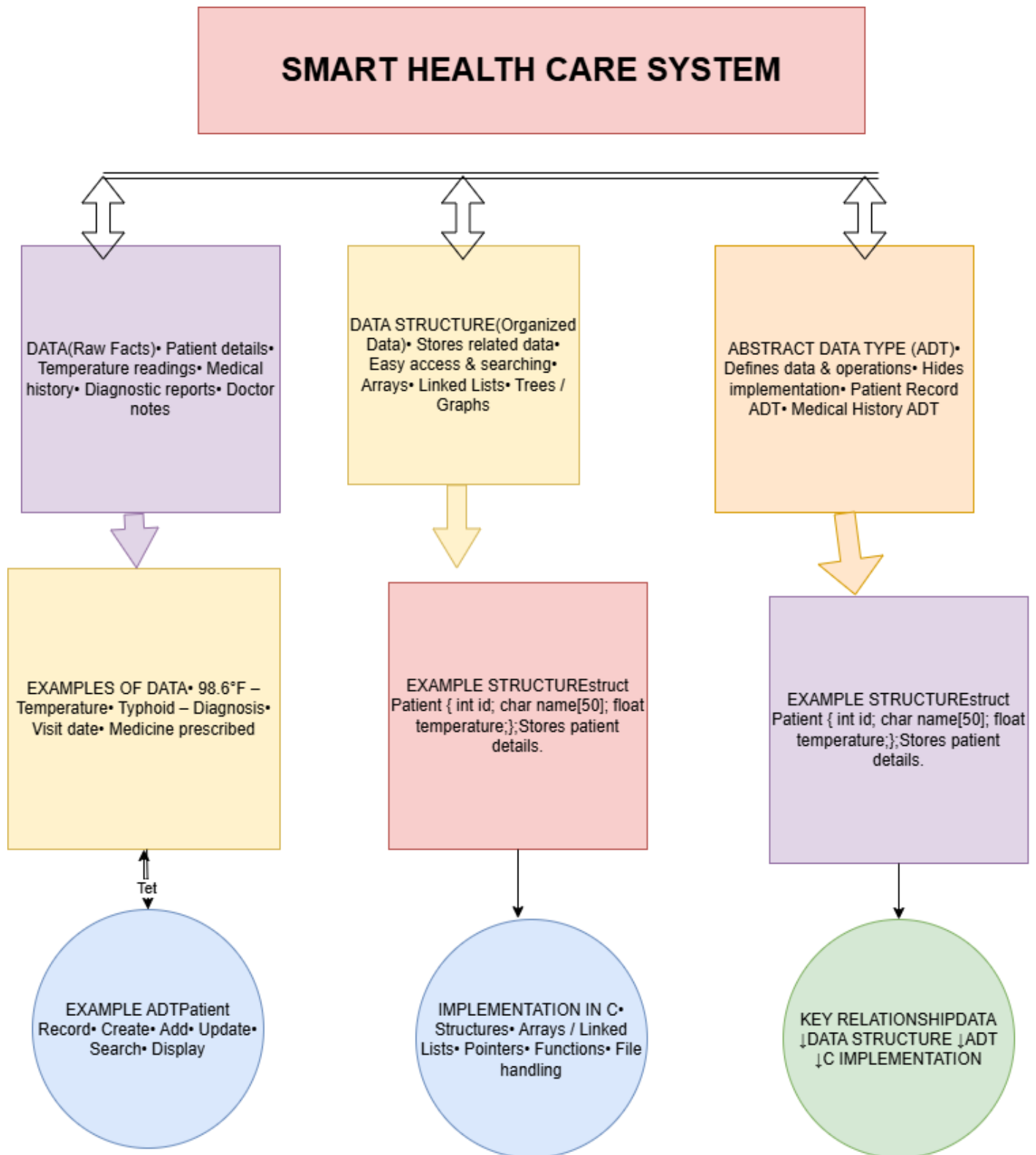
Signature of the student:



RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1.CONCEPT MAPS



2.CONCEPT MAP

SMART CITY TRAFFIC MANAGEMENT SYSTEM

NON-PRIMITIVE DATA STRUCTURES(Build from basic data types)

- Structures – to store complex data like vehicle, route, signal, etc
- Arrays – to store lists of vehicles, roads, signals.
- Linked Lists – for dynamic lists (e.g., vehicles in a lane).
- Trees – for hierarchical data (e.g., city map / road hierarchy)
- Graphs – to represent road networks and shortest paths.

EXAMPLES

- struct Vehicle { ... };
- struct Road { ... };
- struct Signal { ... };
- Graph<ROAD NETWORK>
- Array of vehicles / routes

LINEAR DATA STRUCTURES(Elements in sequence)

- Arrays – fixed size, simple access
- Linked Lists – dynamic, easy insertion/deletion.
- Stacks – for undo/redo, history (e.g., signal state).
- Queues – for managing vehicles at signals or intersections.

EXAMPLE

```
int vehicleQueue[MAX];
// queue to store vehicles
struct Route {
    int routeID;
    char source[50];
    char destination[50];
    int stops;
};
```

NON-LINEAR DATA STRUCTURES(Hierarchical / Network form)

- Trees – for hierarchical data (e.g., city zones, routes).
- Graphs – to model road networks, intersections and shortest path.
- Hash Tables – for fast search (e.g., vehicle ID lookup).
- Heaps – for priority queues (e.g., emergency vehicles).

EXAMPLE

```
struct Node {
    int id;
    struct Node *next;
};
// Used in graphs (road network) or trees (city zones)
```

IMPLEMENTATION IN C

- Define structures for Bus, Route, Vehicle, Passenger, Ticket.
- Use arrays/linked lists for storing records.
- Use graphs for road network and shortest path.
- Use queue/stack for signal and vehicle management.
- File handling to save and load data.

KEY RELATIONSHIPS

NON-PRIMITIVE STRUCTURES
↓ contains
LINEAR STRUCTURES
↓ supports
NON-LINEAR STRUCTURES

APPLICATIONS

- Real-time traffic monitoring
- Route planning and navigation
- Intelligent traffic signals
- Vehicle and passenger tracking
- Emergency vehicle priority
- Smart parking and congestion control

EFFICIENCY AND WHY IT WORKS

1. • Non-primitive structures store complex real-world data efficiently.
2. • Linear structures allow fast access and simple management of sequential data.
3. • Non-linear structures (trees/graphs) are ideal for network and hierarchical data like road maps.
4. • Together, they reduce search time, improve traffic flow, and allow real-time decision making.
5. • This leads to better road safety, less congestion, optimized fuel usage and an overall smart city transport system.



SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

